COMPUTER SYSTEMS AND PLATFORM TECHNOLOGIES

IT0047

FEU ALABANG FEU DILIMAN FEU TECH

Technology Driven by Innovation

MODULE 7.1

Password Protected TASM Progamming

- **At the end of the lesson, the student should be able to :**
- To be able to compare the three character input services of INT 21h.
- The students should be able to create character input and password protected program.
-
- 21

Password Protected TASM Progamming

CALL / RET instruction

Is similar to jump but it allows to return back to the main program.

Syntax :

Call {arguments}

..

Ret

←

Password Protected TASM Progamming

Ex.

. Call down

.

. Down proc

.

.

.

ret

Password Protected TASM Progamming

Sample program:

```
.model small
.stack
.data
  mess db "hello$”
    mess1 db  “goodbye$”
.code
    org 100h
s:
main proc
    mov ax,@data
    mov ds,ax
    mov ah,9
    lea dx,mess
    int 21h
    call down
    mov ah,9
```

Password Protected TASM Progamming

```
    mov dx, offset mess1
    int 21h
    mov ah,4ch
    int 21h
  main endp

        down proc
            mov ah,02
            mov dl,10
            int 21h
            mov dl,13
            int 21h
        ret
        down endp
    end s
```

Password Protected TASM Progamming

JMP instructions

⇒JMP stands for jump

⇒Jump to a label

⇒Allows the programmer to skip sections of a program and branch to any part of the memory for the next instruction

Format:

jmp label

label – refers to the address of an instruction procedure or segment

ex. jmp start

this instruction will transfer the flow of control to the label start

Password Protected TASM Progamming

Ex.

```
.mode small
.code
Org 100h
S:
Mov ah,02
Mov dl,'a'
Int 21h
Jmp x
Mov dl,'b'
Int 21h
X:
    Mov dl,'c'
    Int 21h
Int 20h
End s
```

Password Protected TASM Progamming

Conditional JUMPs

⇒same as JMP instruction but JMP is only taken if the condition is TRUE.

- JA – jump if above = >
- JNA – jump if not above = <=
- JB – jump if below = <
- JNB – jump if not below = >=
- JE – jump if equal = =
- JNE – jump if not equal = !=

Password Protected TASM Progamming

Format:

Conditional_jump level

Ex. je start

<u>CMP instruction</u>

Cmp stands for compare

Compare bytes or words

Used to compare two numeric data fields, one or both of which are contained in a register

Password Protected TASM Progamming

Format: cmp s1,s2

Takes the relation of s1 and s2

Ex.

```
cmp dx,00        ;is dx=00?
        je L10          ;if yes, jump to L10
L10:……              ;jump point if dx = 00
```

Note: cmp instruction is used with conditional jumps

Ex.

```
cmp al, bl
jmp
```

Password Protected TASM Progamming

Sample program:

```
.model small
.code
Org 100h
S:
    Mov ax,03
    Int 10h
  Mov ah,1
  Cmp al,'x'
  Jne exit
  Int 21h
  Exit:
      Int 20h
End s
```

Password Protected TASM Progamming

LOOP

⇒allows a part of a program to be repeated for several times

⇒the number of loops depends upon the value of the CX register

⇒CX takes the number of loops

⇒Each time the loop is encountered the value of CX is decremented

⇒Looping stops when CX becomes zero

Password Protected TASM Progamming

Format:

LOOP label

Ex. LOOP start

Sample program:

```
.model small
.code
Org 100h
S:
Mov ax,03
Int 10h
Mov cx,3
X:
Mov ah,2
Mov dl,'A'
Int 21h
Loop x
        Int 20h
End s
```

Int 21h

MODULE 7.2

Character Input TASM Program

At the end of the lesson, the student should be able to :

- **To be able to compare the three character input services of INT 21h.**
- **The students should be able to create character input and password protected program.**

Character Input Tasm Program

Strings can't usually fit in a register, string are then placed in the memory and then pass the address of the string in memory of two of the registers, the segment address in DS and offset address in DX.

The string output sends a string of characters to the standard output.

On entry: AH = 09h.

DS = segment address of the first character of the string
DX = offset address of the first character of the string

Service 9 display string of character starting with the first character (address in DS: DX) output, but not including, the character "$" (ASCII24H)

Character Input Tasm Program

Int 21h, service 9

display string

ah = 9

[ds:dx] = address of the string

ds = address or location of the string

dx = register to hold the string

Character Input Tasm Program

- **how to declare a string**
-
- **Syntax :**
  - var_name db “string$”
-
- db = define byte
- $ = string terminator
-

Character Input Tasm Program

how to declare a string

Syntax :

var_name db “string$”

db = define byte

$ = string terminator

MODULE 7.3

Cursor Position

At the end of the lesson, the student should be able to :

- To be able to apply the different attribute of INT 10 H in assembly programming
- Identify the attributes used in positioning the display output
- Create a program that will apply the cursor position.

Interrupt 10h Service 06h

INT 10H functions is used to generate a screen attribute and perform such actions as scroll up or down, read attribute or character, or print / display attribute or character.

Interrupt 10h Service 06h

Interrupt 10H of function 02H (video function) enables the programmer to position the cursor in preparation of outputting a character or string.

Interrupt 10h Service 06h

The function set uses the DX register (DH & DL). The row value is placed on the DH register while the column value is placed on the DL register. The row and column ranges are 0-24 and 0-79 respectively.

Interrupt 10h Service 06h

This effectively puts the cursor on the (row value, column value) position. The syntax for using the cursor position function is:

```
mov ah, 02h
mov dh, rr (Row Coordinates)
mov dl,  cc (Column Coordinates)
mov bh, pp (page number)
int 10h
```

Interrupt 10h Service 06h

```
model small
.code
org 100h
start:
mov ah,06h
mov bh,60h
mov ch,0
mov cl,0
mov dh,4
mov dl,79
int 10h

mov ah,06h
mov bh,50h
mov ch,5
mov cl,0
mov dh,9
mov dl,79
int 10h

mov ah,06h
mov bh,40h
mov ch,10
mov cl,0
mov dh,14
mov dl,79
int 10h

mov ah,06h
mov bh,30h
mov ch,15
mov cl,0
mov dh,19
mov dl,79
int 10h

mov ah,06h
mov bh,20h
mov ch,20
mov cl,0
mov dh,23
mov dl,79
int 10h

int 20h

end start
```

MODULE 7.4

Working with Numbers Tasm Program

At the end of the lesson, the student should be able to :

- Understand how math works in assembly language programming.
- Create a program that can perform numeric operations

Character Input Tasm Program

An assembly program consists of a set of statements. The two type of statements are the ***instructions*** and ***directives.***

Instruction:

Ssuch as MOV and ADD, which the assembler that translate to object code

Directives:

Which tell the assembler to perform specific action, such as define a data item.

Character Input Tasm Program

An assembly language supports a number of statements that enable you to control the way in which a source program assembles and list.

An operator provides a facility for changing or analyzing operands during assembly.

Operators are divided into various categories:

Character Input Tasm Program

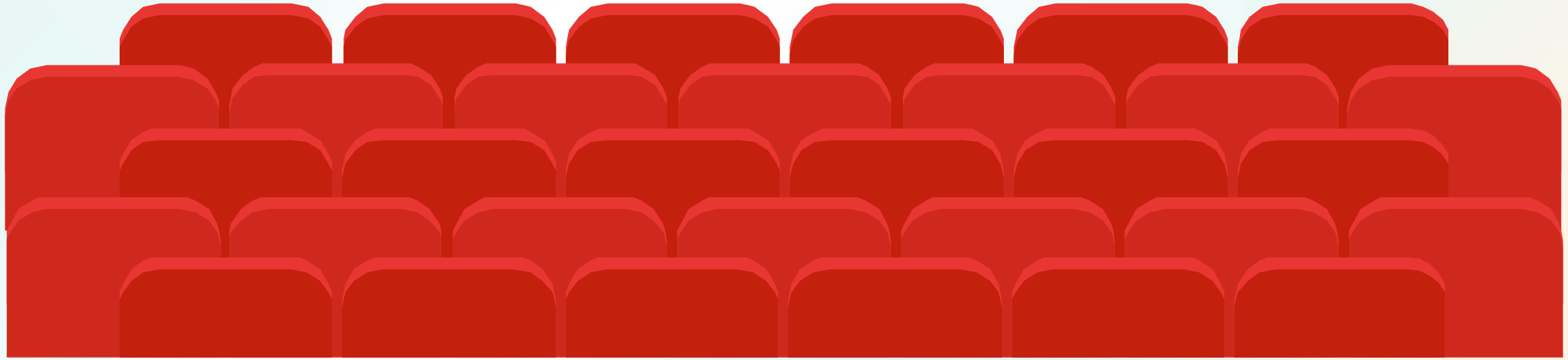
Calculation Operators:	**Arithmetic, index, logic, shift and structure field name.**
Record Operators:	MASK and WIDTH
Relational Operators:	EQ, GE, GT, LE, LT and NE
Segment Operators:	OffSET, SEG, and segment override
Type (or attribute) operators)	HIGH, HIGHWORD, LENGTH, LOW, LOWWORD, PTR, SHORT, SIZE THIS and TYPE.

Character Input Tasm Program

Arithmetic Operators

These operators include the familiar arithmetic signs and perform arithmetic during the assembly. In most cases, you could perform the calculation yourself, although the advantage of using these operators is that every time you change the program and reassemble it, the assembler automatically recalculate with an example or their use.

Q&A SESSION

ASK ANY QUESTION RELATED TO OUR TOPIC FOR TODAY.

THANK YOU